

O S

# 操作系统



O S

# Chapter 5

## 存储管理



O S

1

存储管理的功能

2

分区存储管理

3

页式管理

4

段式与段页式管理

5

覆盖与交换



# 虚拟内存

➤ 由用户编写的源程序，首先要由编译程序编译成CPU可执行的目标代码。然后，链接程序把一个进程的不同程序段链接起来以完成所要求的功能。显然，对于不同的程序段，应具有不同的地址。

# 虚拟内存

➤ 如何安排编译后的目标代码的地址？



# 虚拟内存

- 按照物理存储器中的位置赋予实际物理地址。
- 能装入内存并发执行的进程数将会大大减少
- 对于较大的进程，当其所要求的总内存容量超过内存容量时将无法执行，
- 编译程序必须知道当前内存的空闲部分及其地址，编译程序将非常复杂

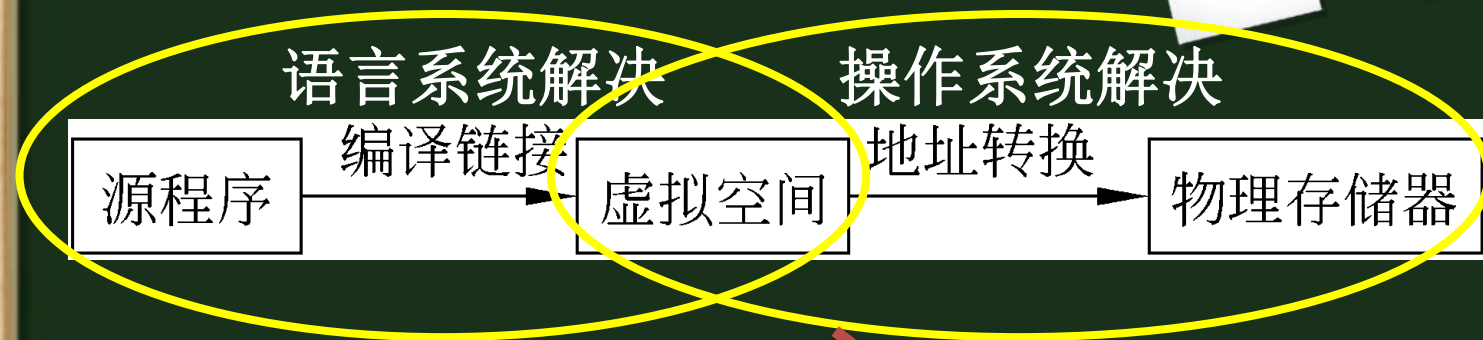
## 解决方案1

# 虚拟内存

- 编译链接程序把用户源程序编译后链接到一个以“0”为始地址的线性或多维**虚拟地址空间**。
- 每个进程都拥有这样一个空间。每个指令或数据单元都在这个虚拟空间中拥有确定的地址（**虚拟地址**）。
- 进程在该空间的地址排列可以是非连续的，其实际物理地址由虚拟地址到实际地址的变换得到

解决方案2

# 虚拟内存



➤ 将进程中的目标代码、数据等的虚拟空间称为**虚拟内存** (virtual memory)。

要实现这个变换，必须要有相应的硬件支持，并使这些硬件能够完成统一管理内存和外存之间数据和程序段自动交换的虚拟存储器功能。

# 虚拟内存

- 虚拟内存不考虑物理存储器的大小和信息存放的实际位置，只规定每个进程中互相关连信息的相对位置。与实际物理存储器数量有限，且被所有进程共享不一样，每个进程都拥有自己的虚拟内存，且虚拟内存的容量是由计算机的地址结构和寻址方式确定的。
- 直接寻址时，如果CPU的有效地址长度为16位，则其寻址范围为0到64K。

# 虚拟内存

➤ 由于每个进程都拥有自己的虚存，且每个虚存的大小不受实际物理存储器的限制，因此，系统不可能提供足够大的内存来存放所有进程的内容。内存中只能存放那些经常被访问的程序和数据段等。这就需要有相当大的外部存储器，以存储那些不经常被访问或在某一段时间内不会被访问的信息。待到进程执行过程中需要这些信息时，再从外存中自动调入主存。



# 地址变换

➤ 内存地址的集合称为内存空间或物理地址空间。内存中，每一个存储单元都与相应的称为内存地址的编号相对应。显然，内存空间是一维线性空间。

# 地址变换

➤ 如何把几个虚拟内存的一维线性空间或多维空间变换到内存唯一的一维物理线性空间

# 地址变换

1

虚拟空间的划分问题

2

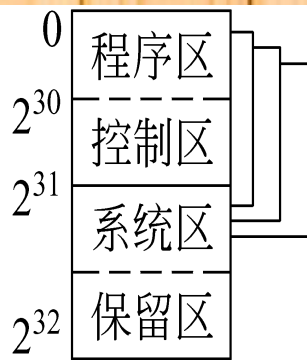
把虚拟空间中已链接和划分好的内容装入内存，并将虚拟地址映射为内存地址的问题，称之为地址重定位或地址映射

# 地址变换---虚拟空间划分问题

- VAX-11型机中的虚拟空间就是划分为**进程空间**和**系统空间**两大部分，而进程空间又更进一步划分为**程序区**和**控制区**。
- VAX-11的虚拟空间容量为 $2^{32}$ 单元，其中程序区占 $2^{30}$ 单元，用来存放用户程序，**程序段以零为基址动态地向高地址方向增长**，最大可达 $2^{30}-1$ 号单元。控制区也占 $2^{30}$ 个单元，存放各种方式和状态下的**堆栈结构及数据**等，其虚拟地址由 $2^{31}-1$ 号地址开始**由高向低地址方向增长**。系统空间占 $2^{31}$ 个单元，用来存放操作系统程序。

进程空间

系统空间



# 地址变换---地址重定位或地址映射

➤地址映射就是要建立虚拟地址与内存地址的关系。  
实现地址重定位或地址映射的方法有两种：**静态地址重定位**和**动态地址重定位**。

在程序调入内存之前由装配程序完成地址映射工作

在程序执行过程中，在CPU访问内存之前，将要访问的程序或数据地址转换成内存地址。动态重定位依靠硬件地址变换机构完成

# 地址重定位或地址映射—静态地址重定位

- 对于虚拟空间内的指令或数据来说，**静态地址重定位**只完成一个首地址不同的连续地址变换。
- 它要求所有待执行的程序必须在执行之前完成它们之间的链接，否则将无法得到正确的内存地址和内存空间。

# 地址重定位或地址映射—静态地址重定位



➤ 不需要硬件支持

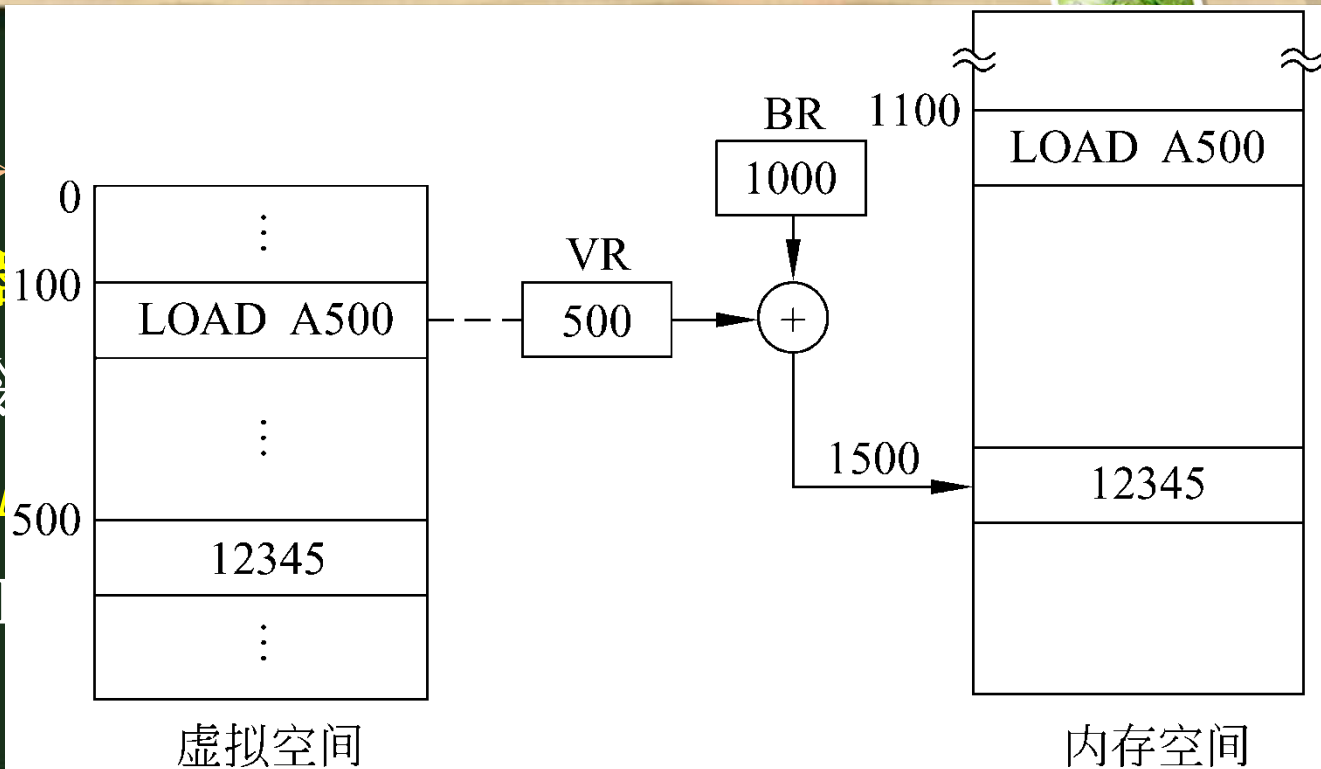


➤ 使用静态重定位方法进行地址变换无法实现虚拟存储器。

➤ 必须占用连续的内存空间，这就难以做到程序和数据的共享。

# 地址重定位或地址映射—动态地址重定位

指令寄存器



寄存器

# 动态地址重定位具体过程



➤ 设置基地址寄存器BR，虚拟地址寄存器VR。



➤ 将程序段装入内存，且将其占用的内存区首地址送BR中。



➤ 在程序执行过程中，将所要访问的虚拟地址送入VR中



➤ 地址变换机构把VR和BR的内容相加，得到实际访问的物理地址。

# 动态地址重定位的优点

## 优点1

➤可以对内存进行非连续分配。

## 优点2

➤动态重定位提供了实现虚拟存储器的基础

## 优点3

➤有利于程序段的共享

# 内外存数据传输的控制

- 要实现内存扩充，在程序执行过程中，内存和外存之间必须经常地交换数据。
- 按什么样的方式来控制内存和外存之间的数据流动呢？



# 内外存数据传输的控制

## 用户程序自己控制

➤ **覆盖技术**要求用户清楚地了解程序的结构，并指定各程序段调入内存的先后次序。覆盖是一种早期的主存扩充技术。使用覆盖技术，**用户负担很大，且程序段的最大长度仍受内存容量限制**。因此，覆盖技术不能实现虚拟存储器。

## 操作系统控制方式

- 交换(swapping)方式
- 请求调入(on demand)方式
- 预调入(on prefetch)方式。

# 内外存数据传输

## 交换方式

➤ 由操作系统把那些

请求调入方式和预调入方式  
可以实现进程大小不受内存  
容量限制的虚拟存储器。

的进程换入内存。

交换方式一般**不进行部分交换**，即每次交换都交换那些除去常驻内存部分后的**整个进程**，而且，即使能完成部分交换，也不是按照执行的需要而交换程序段，只是把受资源限制、暂时不能执行的程序段换出内存。因此，虽然交换方式也能完成内存扩充任务，但它仍未实现那种所谓自动覆盖、内存和外存统一管理、进程大小不受内存容量限制的虚拟存储器。

动地从外存将有关的程序段和数据段调入内存的一种操作系统控制方式。

➤ 而预调入则是由操作系统预测在不远的将来会访问到的那些程序段和数据段部分，并在它们被访问之前系统选择适当的时机将它们调入内存的一种数据流控制方式。



**分配结构：**登记内存使用情况，供分配程序使用的表格与链表。例如内存空闲区表、空闲区队列等。

**放置策略：**确定调入内存的程序和数据在内存中的位置。这是一种选择内存空闲区的策略。

**交换策略：**在需要将某个程序段和数据调入内存时，如果内存中没有足够的空闲区，由交换策略来确定把内存中的哪些程序段和数据段调出内存，以便腾出足够的空间。

**调入策略：**外存中的程序段和数据段什么时间按什么样的控制方式进入内存。调入策略与5.1.3节中所述内外存数据流动控制方式有关。

**回收策略：**回收策略包括二点，一是回收的时机，二是对所回收的内存空闲区和已存在的内存空闲区的调整。

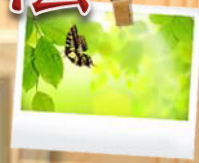
# 内存信息的共享与保护

➤在多道程序设计环境下，内存中的许多用户或系统程序和数据段可供不同的用户进程共享。这种资源共享将会提高内存的利用率。但是，反过来说，除了被允许共享的部分之外，又要**限制各进程只在自己的存储区活动**，各进程**不能**对别的进程的程序和数据段产生**干扰和破坏**，因此须对内存中的**程序和数据段采取保护措施**。

# 常用的内存信息保护方法



# 内存信息的共享与保护---上下界保护法



上界寄存器 UR

100K

100K

⋮

被保护  
程序

200K

下界寄存器 LR

200K

⋮

内存

$100K \leq \text{被访问地址} \leq 200K$

➤上下  
寄存器  
始地址  
访问操  
位后的  
内。若  
法的，

下界  
的起  
进行  
重定  
固之  
是非

# 内存信息的共享与保护---保护键法



保护键法为每一个被保护的存储块分配一个单独

当前程序状态字

|     |   |     |
|-----|---|-----|
| ... | 2 | ... |
|-----|---|-----|

开关字节

|    |   |   |   |   |
|----|---|---|---|---|
| 2K | ⋮ |   |   |   |
| 4K |   | 0 | R | W |
| 6K |   | 2 | W |   |
|    | ⋮ |   |   |   |

内存

正确访问

LOAD 1 5000 非读保护

STORE 2 5200 开关字-键匹配

非正确访问

LOAD 1 2500 出错, 开关字-键不匹配

的。

# 内存信息的共享与保护---相结合

- 在这种保护模式下，**用户态进程**只能访问那些在**界限寄存器**所规定范围内的内存部分，而**核心态进程**则可以访问整个内存地址空间。
- UNIX系统就是采用的这种内存保护方式。

O S

1

存储管理的功能

2

分区存储管理

3

页式管理

4

段式与段页式管理

5

覆盖与交换技术



# 分区管理的基本原理

- 分区管理的基本原理是给每一个内存中的进程划分一块适当大小的存储区，以连续存储各进程的程序和数据，使各进程得以并发执行。
- 固定分区法+动态分区法

# 分区管理---固定分区法

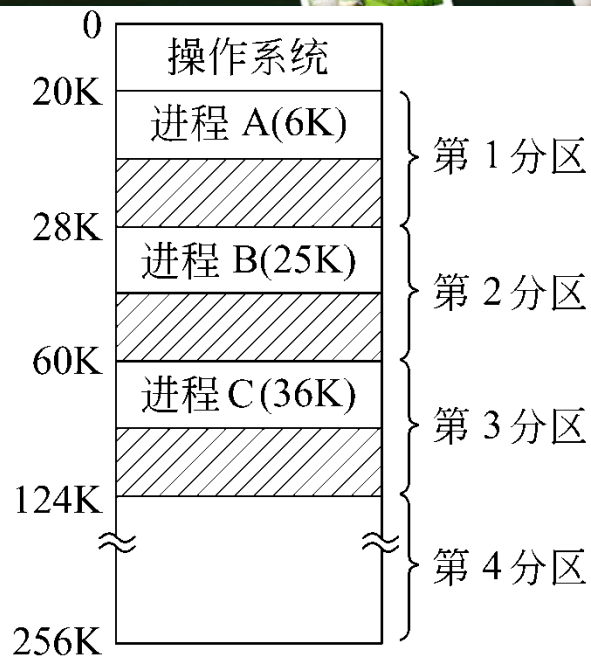
- 固定分区法就是把内存区固定地划分为若干个大小不等的区域。分区划分的原则由一般系统操作员或操作系统决定。
- 例如可划分为长作业分区和短作业分区。分区一旦划分结束，在整个执行过程中每个分区的长度和内存的总分区个数将保持不变。



# 分区管理---固定分区法

| 区号 | 分区长度 | 起始地址 | 状态  |
|----|------|------|-----|
| 1  | 8K   | 20K  | 已分配 |
| 2  | 32K  | 28K  | 已分配 |
| 3  | 64K  | 60K  | 已分配 |
| 4  | 132K | 124K | 未分配 |

(a) 分区说明表



(b) 内存状态

# 分区管理---固定分区法



➤ 存在即使是小作业也要占据大分区的浪费现象

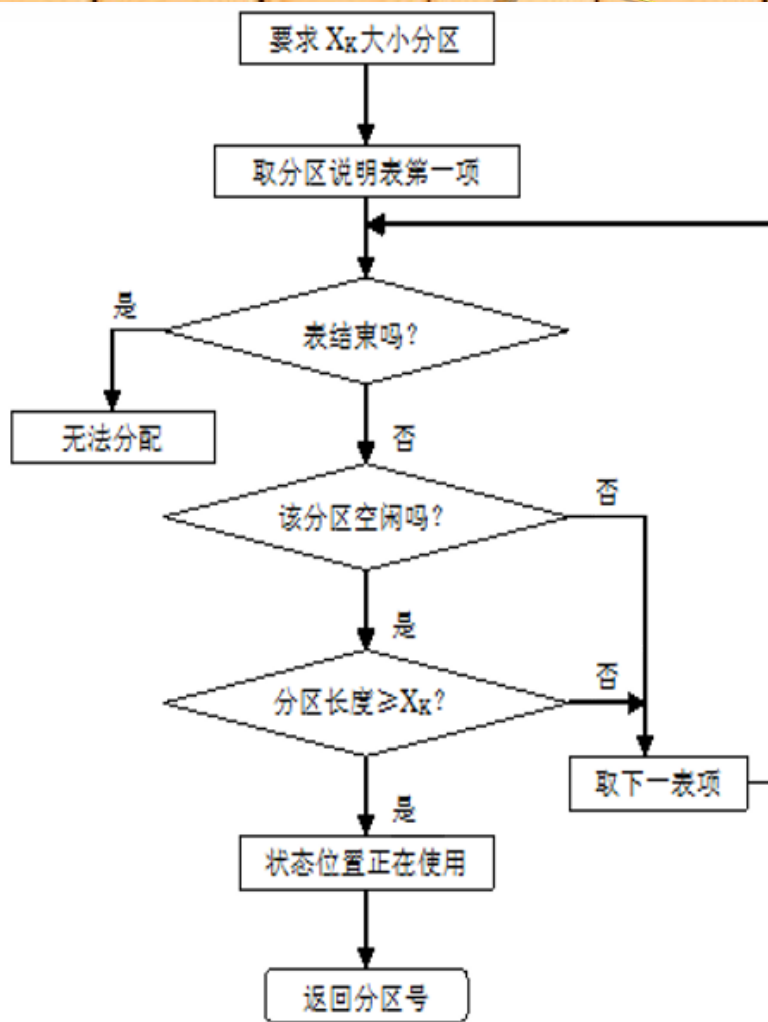


# 固定分区时的分配与回收

➤ 固定分区的回收更加简单。当进程执行完毕，不再需要内存资源时，管理程序将对应的分区状态置为未使用即可。

# 固定分区

➤ 用户程序要求  
分配要求和  
程序根据请求  
足要求的空间



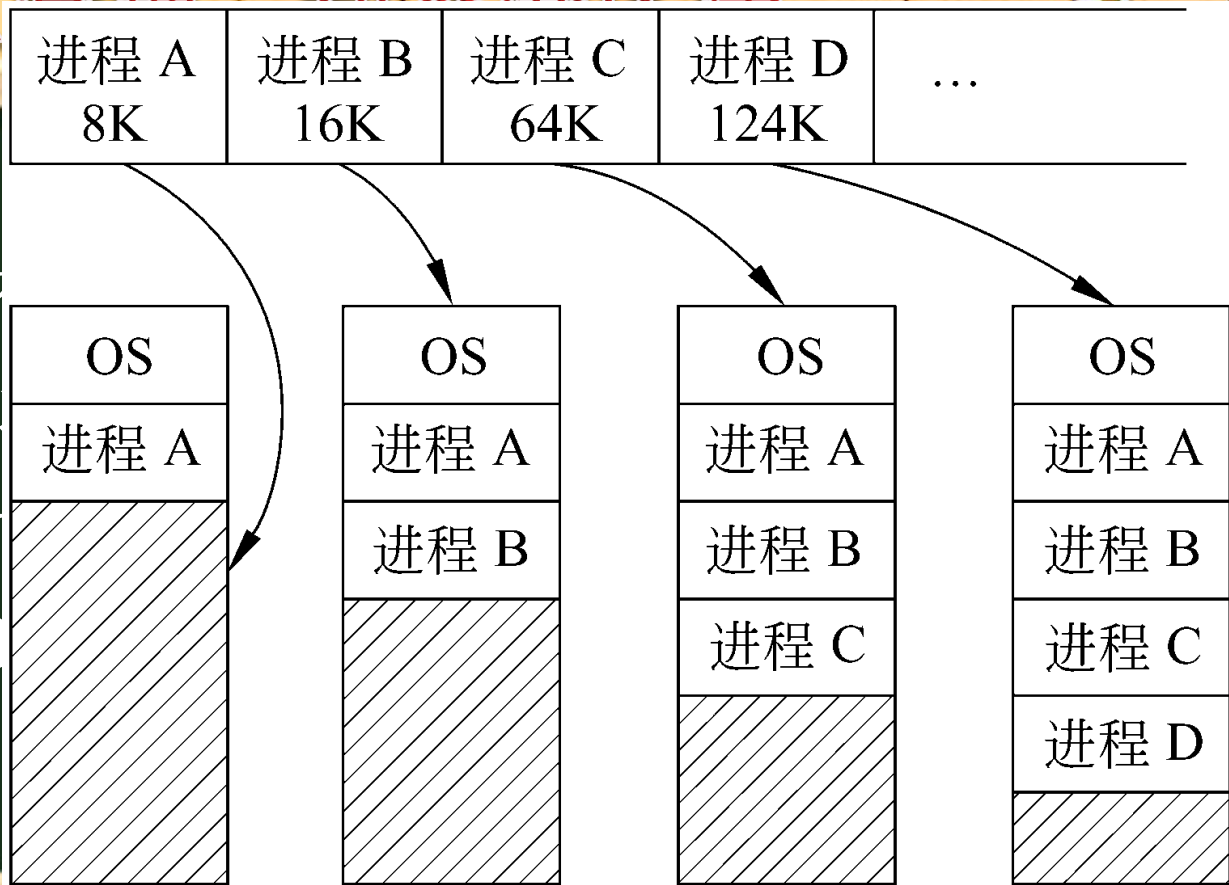
出内存  
管理程  
一个满

# 分区管理---动态分区法

- 动态分区法在作业执行前并不建立分区，分区的建立是在作业的处理过程中进行的，且其大小可随作业或进程对内存的要求而改变。
- 提高了内存的利用率

# 分区管理 动态分区法

➤ 分区管理  
➤ 动态分区法



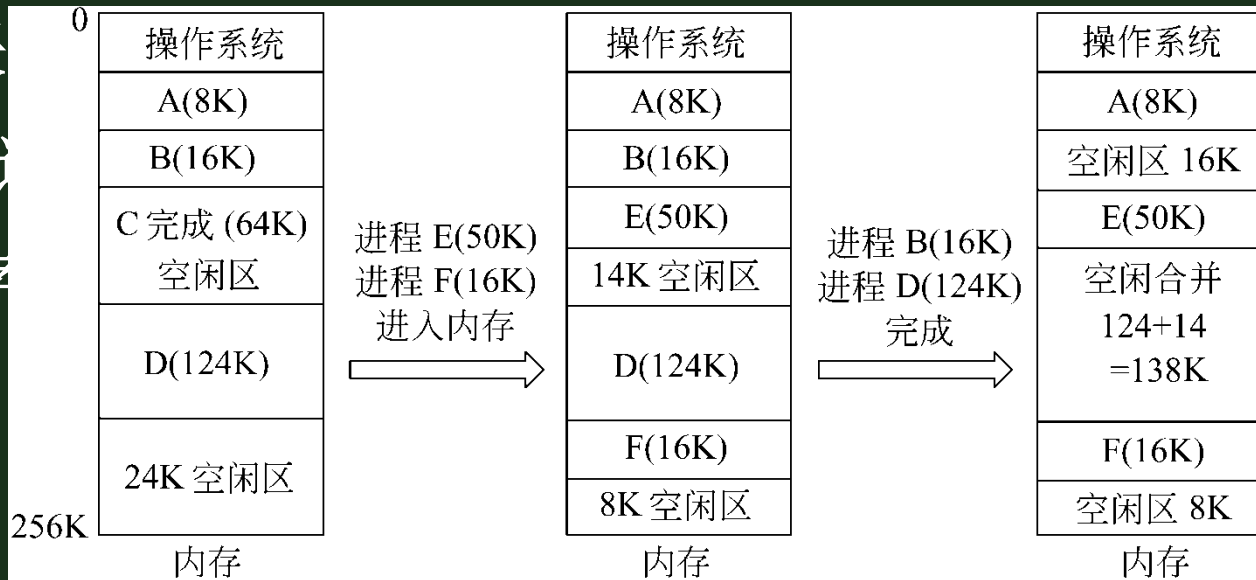
# 分区管理---动态分区法

➤随着进程的执行，会出现一系列的分配和**释放**。如在某一时刻，进程**C执行结束**并释放内存之后，管理程序又要为另两个**进程E**（设需内存50K）和**F**（设需内存16K）**分配内存**。如果分配的空闲区比所要求的大，则管理程序将该空闲区分成两个部分，其中一部分成为已分配区而另一部分成为一个小空闲区。



# 分区管理---动态分区法

➤ 采用  
E和  
内存

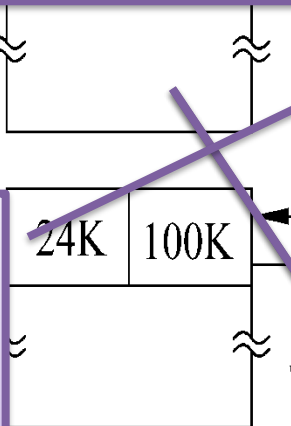


# 分区管理---动态分区

动态分区说明表的每个表目记录一个空闲区，主要参数包括区号、长度和起始地址。采用表格结构，管理过程比较简单，但表的大小难以确定，可用表要占用一部分内存。

| 区号 | 分区长度 | 起始地址 |
|----|------|------|
| 1  | 16K  | 40K  |
| 3  | 24K  | 78K  |

| 请求长度           |     |
|----------------|-----|
| P <sub>1</sub> | 13K |
| P <sub>2</sub> | 20K |



(b) 自由链

查找时要比可用表困难，但由于自由链指针是利用的空闲区自身的单元，所以不必占用额外的内存区

(c) 请求表

利用每个内存空闲单元存放本空闲区的起始地址，系统再设置一指针指向第一个空闲区

请求表的每个表目描述请求内存资源的作业或进程号以及所请求的内存大小。

# 动态分区时的分配与回收

➤ 主题

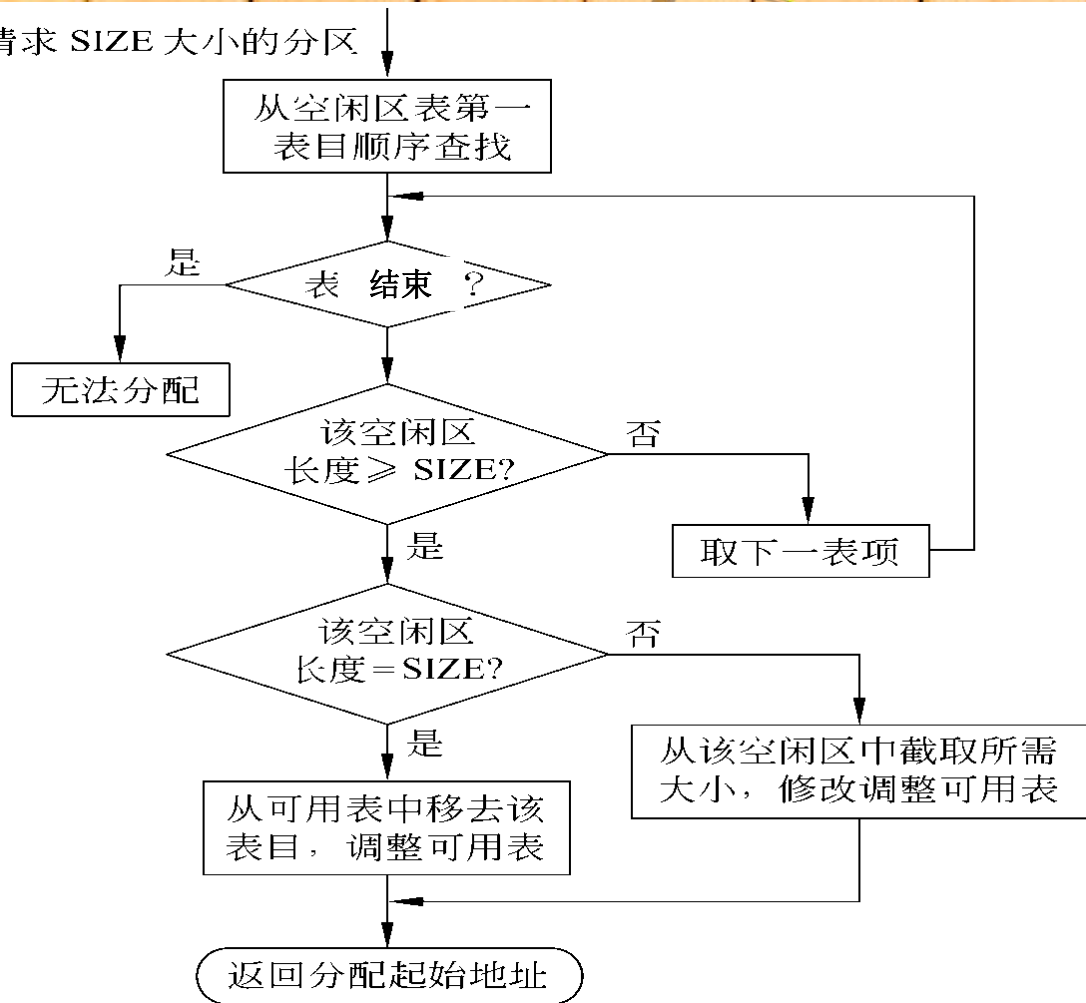
对于请求表中的要求内存长度，从可用表或自由链中寻找出合适的空闲区分配程序。

分配空闲区之后，更新可用表或自由链

进程或作业释放内存资源时，和相邻的空闲区进行链接合并，更新可用表或自由链。

# 动态分

请求 SIZE 大小的分区



➤要求可  
该算法的  
存长度的  
到的分区  
把余下的  
留在可用

非列。  
文内  
寻找  
并后

# 动态分区分配方法---最佳适应算法

➤要求从小到大的次序组成空闲区可用表或自由链。  
当用户作业或进程申请一个空闲区时，存储管理程序从表头开始查找，当找到第一个满足要求的空闲区时，停止查找。如果该空闲区大于请求表中的请求长度，则与最先适应法时相同，将减去请求长度后的剩余空闲区部分留在可用表中。

# 动态分区分配方法---最坏适应算法

➤要求从大到小的次序组成空闲区可用表或自由链。  
当用户作业或进程申请一个空闲区时，先检查空闲区可用表或自由链的第一个空闲可用区的大小是否大于或等于所要求的内存长度，若不满足则分配失败，若满足要求的空闲区，则减去请求长度后的剩余空闲区部分留在可用表中。

# 几种分配算法的比较

➤ **搜索速度：最先适应算法最佳。** 尽管最佳适应算法或最坏适应算法看上去能很快找到一个最适合的或最大的空闲区，但后两种算法都要求首先把不同大小的空闲区按其大小进行排队，这实际上是对所有空闲区进行一次搜索。



# 几种分配算法的比较

- 最先适应算法尽可能地利用低地址空间，从而保证高地址有较大的空闲区来放置要求内存较多的进程或作业。
- 最佳适应法找到的空闲区最佳，用最佳适应法找到的空闲区或者是正好等于用户请求的大小或者是能满足用户要求的最小空闲区。尽管最佳适应法能选出最适合用户要求的可用空闲区，但这样做在某些情况下并不一定能提高内存的利用率。

# 几种分配算法的比较

➤最坏适应算法正是基于不留下碎片空闲区这一出发点的。它选择最大的空闲区来满足用户要求，以期分配后的剩余部分仍能进行再分配。



# 动态分区时的回收与拼接

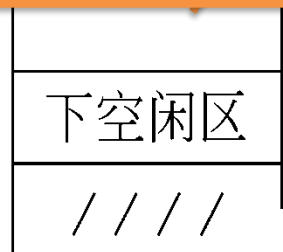
➤也涉及**剩余空闲区拼接问题**。如果不对空闲区进行拼接，则由于每个作业或进程所要求的内存长度不一样而出现大量分散，较小的空闲区。这就造成大量的内存浪费。



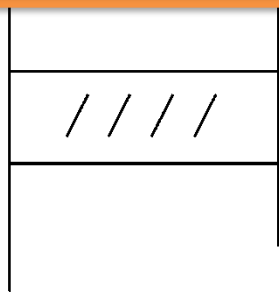
将释放区与下空闲区合并，并将释放区的起始地址作为合并区的起始地址。合并区的大小为上空闲区与释放区之和。同时，修改上空闲区对应的表项或自由链指针。

释放区作为一个新可用区插入可用表或自由链。同学可以自行写出动态分区时的回收算法。

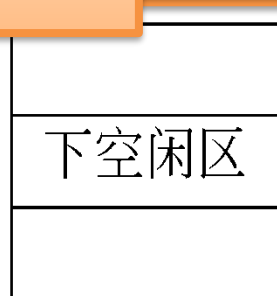
将释放区与上空闲区合并为一个空闲区，其起始地址为上空闲区的起始地址。合并后，修改上空闲区对应的表项或自由链指针。



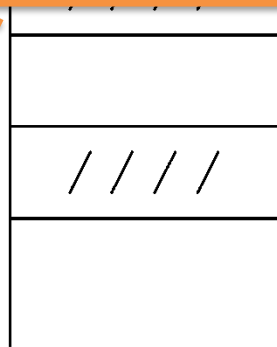
(a) 上下相邻区都是空闲区



(b) 上相邻区为空闲区



(c) 下相邻区为空闲区



(d) 上下相邻区都不是空闲区

# 几种分配算法的比较

➤ **回收过程：最先适应算法最佳。**因为使用最先适应算法回收某一空闲区时，无论被释放区是否与空闲区相邻，都不用改变该区在可用表或自由链中的位置，只需修改其大小或起始地址。而最佳适应算法和最坏适应算法都必须重新调整该区的位置。



# 分区管理的主要优点



➤ 实现了多个作业或进程对内存的共享，有助于多道程序设计，从而提高了系统的资源利用率。

➤ 该方法要求的硬件支持少，管理算法简单，因而实现容易。



# 分区管理的主要缺点



- **内存利用率仍然不高。**和单一连续分配算法一样，存储器中可能含有从未用过的信息。而且，还存在着严重的碎小空闲区(碎片)不能利用的问题，这更进一步影响了内存的利用率。
- **无法实现各分区间的信息共享。**
- **作业或进程的大小受分区大小控制，**除非配合采用覆盖和交换技术。



O S

1

存储管理的功能

2

分区存储管理

3

页式管理

4

段式与段页式管理

5

覆盖与交换技术



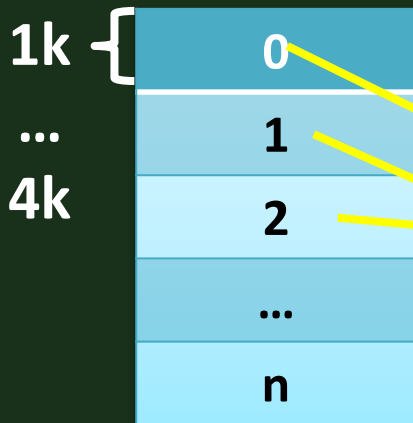
# 页式管理的基本原理

- 分区管理存在缺陷
- 页式管理正是为了减少碎片以及为了只在内存存放那些反复执行或即将执行的程序段与数据部分，而把那些不经常执行的程序段和数据存放于外存待执行时调入，以提高内存利用率而提出来的。



# 页式管理的基本原理

➤ 各进程的虚拟空间被划分成若干个长度相等的页 (page)



➤ 把内存空间也按页的大小划分为片或页面 (page frame)



➤ 进程的虚地址=页号P+页内地址W

# 页式管理的基本原理

- 用户进程在内存空间内除了在每个**页面内地址连续**之外，每个**页面之间不再连续**。
- 第一实现了**内存中碎片的减少**，因为任一碎片都会小于一个页面。
- 第二是实现了**由连续存储到非连续存储这个飞跃**，为在内存中局部地、动态地存储那些反复执行或即将执行的程序和数据段打下了基础。

# 页式管理的基本原理

➤ 怎样由页式虚拟地址变换为  
内存页面物理地址呢？



# 页式管理的基本原理

➤ 把页式虚地址与内存页面物理地址建立一一**对应页表**，并用相应的**硬件地址变换机构**，来解决离散地址变换问题。页表方式实质上是动态重定位技术的一种延伸。

# 页式管理的基本原理

- 页式管理采用**请求调页**或**预调页技术**实现了内外存储区的统一管理。
- 内存中只放经常被执行或即将被执行的页，其他的则存放在外存中需要时再调入。

静态页  
式管理



动态页  
式管理

# 静态页式管理

➤ 静态页面管理方法是在作业或进程**开始执行之前**，把该作业或进程的程序段和数据**全部装入内存**的各个页面中，并通过**页表** (page mapping table) 和**硬件地址变换机构**实现虚拟地址到内存物理地址的地址映射。

# 静态页式管理

- 静态分页管理的第一步是为要求内存的作业或进程分配足够的页面。
- 系统依靠**存储页面表**、**请求表**以及**页表**来完成内存的分配工作。

# 静态—内存页面分配与回收—页表

➤ 页表：最简单的页表

由页号与页面号组成。

| 页号 | 页面号 |
|----|-----|
|    |     |
|    |     |
|    |     |

➤ 页表在内存中占有一块固定的存储区。页表的大小由进程或作业的长度决定。

➤ 页式管理时每个进程至少拥有一个页表。

# 静态—内存页面分配与回收—请求表

➤ 用来确定作业或进程的虚拟空间的各页在内存中

| 进程号   | 请求页面数 | 页表始址  | 页表长度  | 状态    |
|-------|-------|-------|-------|-------|
| 1     | 20    | 1024  | 20    | 已分配   |
| 2     | 34    | 1044  | 34    | 已分配   |
| 3     | 18    | 1078  | 18    | 已分配   |
| 4     | 21    | ..... | ..... | 未分配   |
| ..... | ..... | ..... | ..... | ..... |

➤ 请求表整个系统一张

# 静态—内存页面分配与回收—存储页面表



出的内存表位置1，

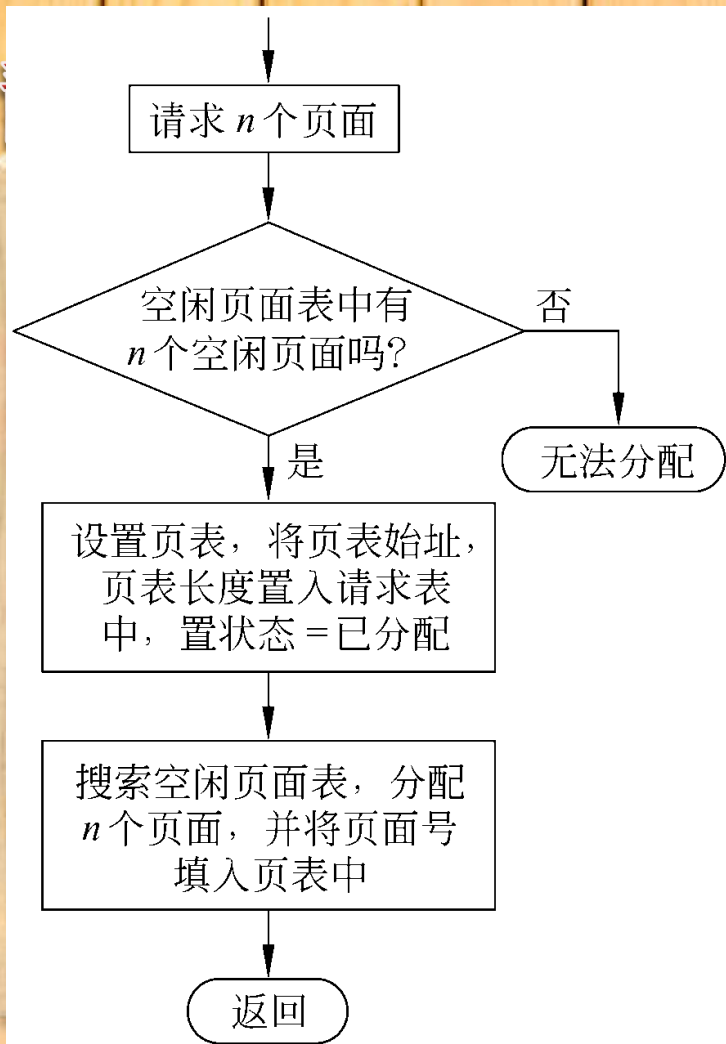
| 19 | 18 | 17 | 16 | 15 | ..... | 4 | 3 | 2 | 1 | 0 |
|----|----|----|----|----|-------|---|---|---|---|---|
| 0  | 1  | 0  | 0  | 1  | ..... | 0 | 1 | 1 | 0 | 1 |
| 0  | 0  | 1  | 0  | 0  | ..... | 1 | 1 | 0 | 1 | 1 |
| 0  | 0  | 0  | 1  | 0  | ..... | 1 | 1 | 1 | 0 | 1 |
| 0  | 0  | 0  | 0  | 1  | ..... | 0 | 0 | 1 | 1 | 1 |
|    |    |    |    |    |       |   |   |   |   |   |

指面在代位

否则直0。这种方法称为位示图法。

# 静态—内存页面分配与回收—存储页面表

➤ 存储页面表的另一种构成办法是采用**空闲页面链**的方法。在空闲页面链中，**队首页面的第一个单元和第二个单元**分别放入**空闲页面总数**与指向下一个空闲页面的**指针**。其他页面的**第一个单元**中则分别放入指向下一个页面的**指针**。空闲页面链的方法由于使用了空闲页面本身的单元存放指针，因此**不占据额外的内存空间**。



|    |    |    |    |    |       |   |   |   |   |   |
|----|----|----|----|----|-------|---|---|---|---|---|
| 19 | 18 | 17 | 16 | 15 | ..... | 4 | 3 | 2 | 1 | 0 |
| 0  | 1  | 0  | 0  | 1  | ..... | 0 | 1 | 1 | 0 | 1 |
| 0  | 0  | 1  | 0  | 0  | ..... | 1 | 1 | 0 | 1 | 1 |
| 0  | 0  | 0  | 1  | 0  | ..... | 1 | 1 | 1 | 0 | 1 |
| 0  | 0  | 0  | 0  | 1  | ..... | 0 | 0 | 1 | 1 | 1 |
|    |    |    |    |    |       |   |   |   |   |   |

| 进程号   | 请求页面数 | 页表始址  | 页表长度  | 状态    |
|-------|-------|-------|-------|-------|
| 1     | 20    | 1024  | 20    | 已分配   |
| 2     | 34    | 1044  | 34    | 已分配   |
| 3     | 18    | 1078  | 18    | 已分配   |
| 4     | 21    | ..... | ..... | 已分配   |
| ..... | ..... | ..... | ..... | ..... |

| 页号 | 页面号 |
|----|-----|
|    |     |
|    |     |
|    |     |

# 静态—地址变换

➤ 怎样由页号和页内相对地址  
变换到内存物理地址？

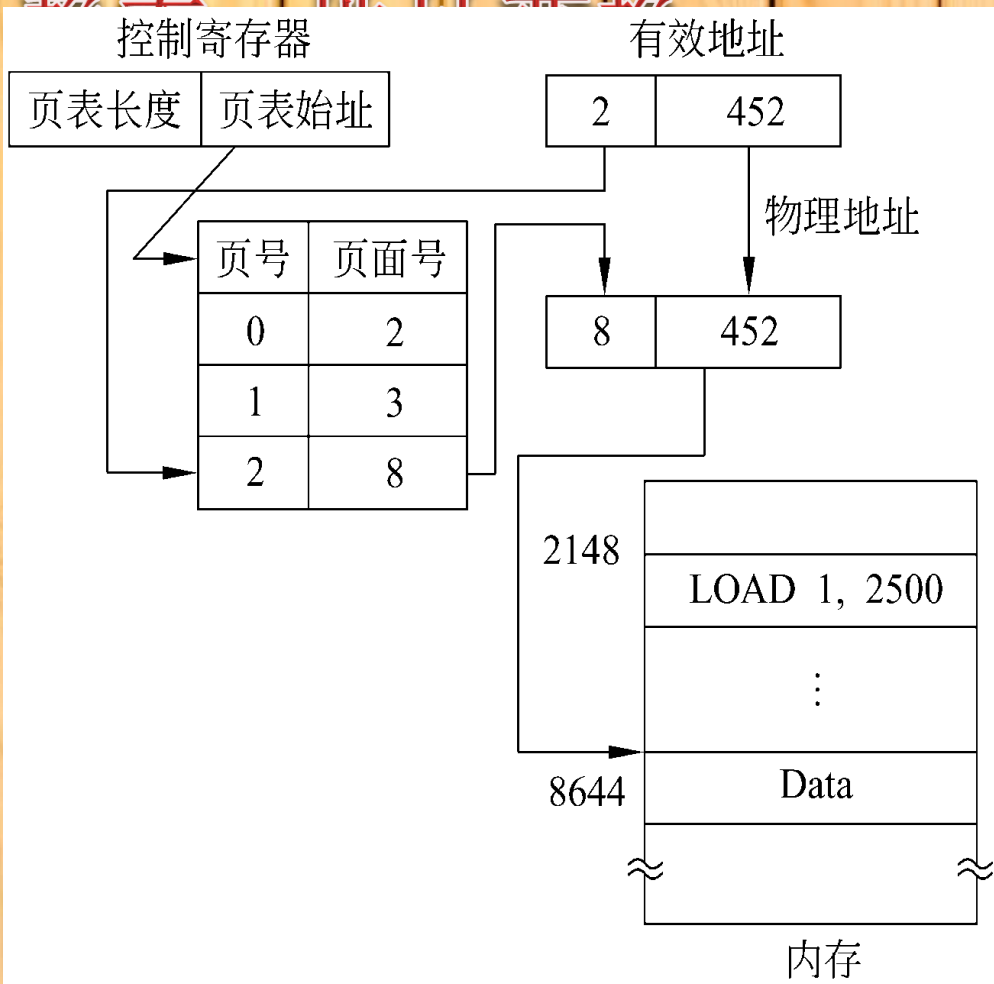


# 静态—地址变换

➤ 由地址分配方式知道，在一个作业或进程的页表中，连续的页号对应于不连续的页面号。例如，设一个3页长的进程具有页号0，1，2，但其对应的页面号则为2，3，8。

➤ 设每个页面长度为1K，指令LOAD 1，2500的虚地址为100

| 页号 | 面号 |
|----|----|
| 0  | 2  |
| 1  | 3  |
| 2  | 8  |



➤ 上述地址变换过程全部由硬件地址变换机构自动完成。

# 静态—地址变换

- 怎样由页号和页内相对地址变换到内存物理地址？
- 页式管理时，地址变换的速度怎样呢？



# 静态—地址变换

➤ 由于页表是驻留在内存的某个固定区域中，而取数据或指令又必须经过页表变换才能得到实际物理地址。因此，**取一个数据或指令至少要访问内存两次以上**。一次访问页表以确定所取数据或指令的物理地址，另一次是根据地址取数据或指令。这比通常执行指令的速度**慢了一倍**。



# 静态—地址变换

➤ 如何提高查找速度？

➤ 一个最直观的办法就是把页表放在寄存器中而不是内存中，但由于寄存器价格太贵，这样做是不可取的。

# 静态—地址变换

- 如何提高查找速度?
- 另一种办法是在地址变换机构中加入一个高速联想存储器，构成一张快表。在快表中，存入那些当前执行进程中最常用的页号与所对应的页面号，从而提高查找速度。



# 静态页式管理



➤ 解决了分区管理时的碎片问题



➤ 由于静态页式管理要求进程或作业在执行前全部装入内存，如果可用页面数小于用户要求时，该作业或进程只好等待。而且，作业或进程的大小仍受内存可用页面数的限制。



# 动态页式管理

➤方法1：请求页式管理的调入方式是，当需要执行某条指令而又发现它不在内存时或当执行某条指令需要访问其他的数据或指令时，这些指令和数据不在内存中，从而发生缺页中断，系统将外存中相应的页面调入内存。



# 动态页式管理

➤方法2：预调入页式管理：系统对那些在外存中的页进行调入顺序计算，估计出这些页中指令和数据的执行和被访问的顺序，并按此顺序将它们顺次调入和调出内存。除了在调入方式上请求页式管理和预调入管理有些区别之外，其他方面这两种方式基本相同。

# 动态页式管理

- 由于请求页式管理只让进程或作业的部分程序和数据驻留在内存中，因此，在执行过程中，不可避免地会出现某些虚页不在内存中的问题。
- 怎样发现这些不在内存中虚页以及如何处理这种情况，是请求页式管理必须解决的两个基本问题。

# 动态页式管理

可以用扩充页表的方法解决。即与每个虚页号相对应，除了页面号之外，再增设该页是否在内存的中断位以及该页在外存中的副本起始地址。

## 问题1

| 页号 | 页面号 | 中断位 | 外存始址 |
|----|-----|-----|------|
| 0  |     |     |      |
| 1  |     |     |      |
| 2  |     |     |      |
| 3  |     |     |      |

# 动态页式管理

## 问题2

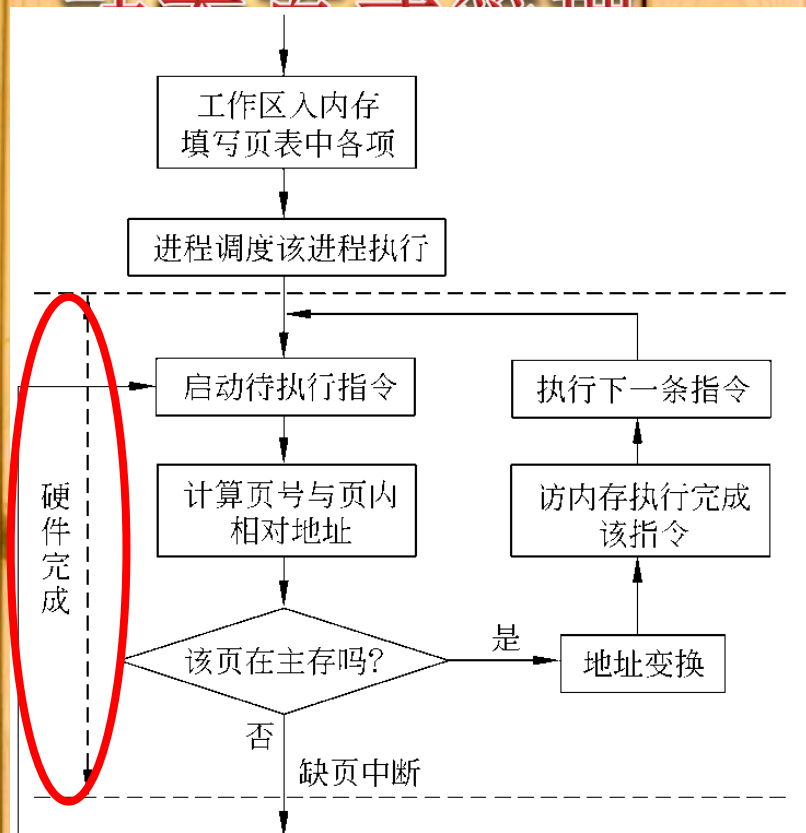
第一，采用何种方式把所缺的页调入内存。  
第二，如果内存中没有空闲页面时，把调进来的页放在什么地方。也就是说，采用什么样的策略来淘汰已占据内存的页

➤如果在内存中的某一页被淘汰，且该页曾因程序的执行而被修改，则显然该页是应该重新写到外存上加以保存的。而那些未被访问修改的页，因为外存已保留有相同的副本，写回外存是没有必要的。因此，在页表中还应增加一项以记录该页是否曾被改变。

# 动态页式管理

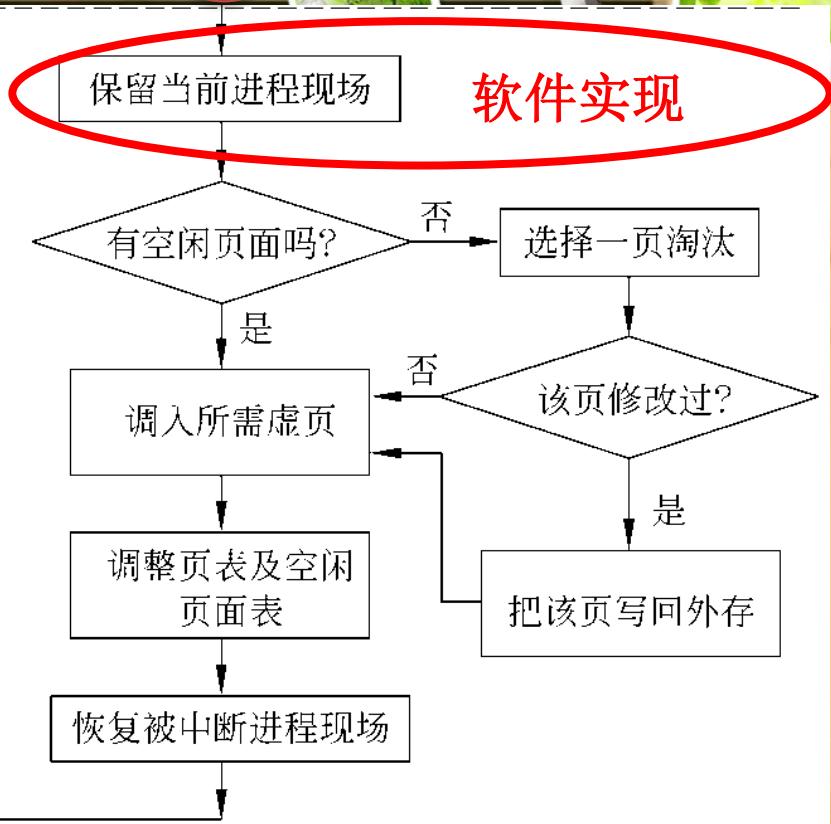
| 页号 | 页面号 | 中断位 | 外存始址 | 改变位 |
|----|-----|-----|------|-----|
|    |     |     |      |     |
|    |     |     |      |     |
|    |     |     |      |     |
|    |     |     |      |     |

# 请求页式管理



A

A



软件实现

# 动态页式管理

➤有关缺页的调入和存放，在内存中没有空闲页面时，实际上是一个内存页面置换算法的问题。选择什么样的**置换算法**，将直接影响到内存利用率和系统效率。

# 动态页式管理

➤事实上，如果置换算法选择不当，有可能产生刚被调出内存的页又要马上被调回内存，调回内存不久又马上被调出内存，如此反复的局面。这使得整个系统的页面调度非常频繁，以致大部分时间都花费在主存和辅存之间的来回调入调出上。这种现象被称为**抖动(thrashing)现象**。

# 请求页式管理中的置换算法

- 在系统设计人员认为无法确定哪些页被访问的概率较低时，随机地选择某个用户的页面并将其换出将是一种明智的作法

## 随机淘汰算法

## 轮转（RR）法

- 循环换出内存可用区内一个可以被换出的页，无论该页是刚被换进或已换进内存很长时间

- 选择在内存驻留时间最长的一页将其淘汰。**FIFO**算法认为先调入内存的页不再被访问的可能性要比其他页大，因而选择最先调入内存的页换出

## 先进先出（FIFO）算法

- 淘汰在访问串中将来再也不出现的或是在当前页最远的位置上出现的页

## 理想型淘汰算法

## 最近未使用（LRU）

- 选择离当前时间最近的一段时间内最久没使用过的页先淘汰

# 请求页式管理中的置换算法--FIFO

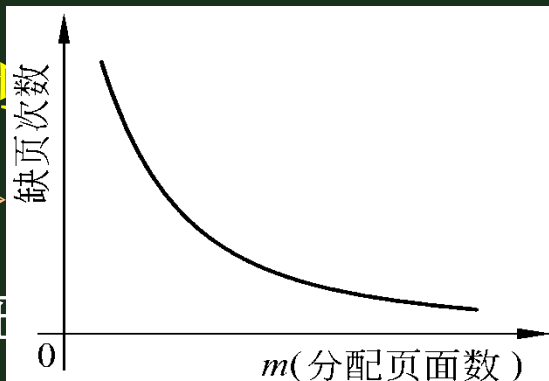
➤ 由实验和测试发现FIFO算法和RR算法的内存

利

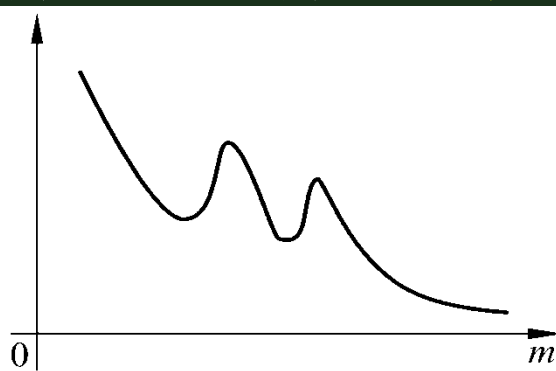
➤

在

有



(a) 正常情况



(b) Belady 现象

加的奇怪现象。这种现象称为Belady现象。

象。

增

# 请求页式管理中的置换算法

进程在一个执行过程中，实际上发生了**12**次缺页。如果设缺页率为缺页次数与访问串中的访问次数之比，  
则该例中的缺页率为 $12/17=70.5\%$

➤例：设进程P共访问17次，

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 2 | 0 | 1 |
| 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   | 0 | 0 | 0 | 0 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 1 | 1 | 1 | 1 |
|   |   | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 2 | 2 | 2 |

内存中有n个进程P所建的程序和数据的各页面变化  
情况如下图所示。

# 请求页式管理中的置换算法--FIFO

➤如果给进程P分配4个物理页面，其物理页面号分别为0、1、2、3，则在进程P执行过程中，物理页面的变化情况如下：

进程在一个执行过程中，实际发生了9次缺页，其缺页率为 $9/17=52.9\%$

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 2 | 0 | 1 |
| 7 | 7 | 7 | 7 | 7 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 2 | 2 | 2 |
|   | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |
|   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   |   |   | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 1 | 1 | 1 | 1 |

# 请求页式管理中的置换算法--FIFO

- 设进程P可分为5页，访问串为1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5。进程在一个执行过程中，实际上发生了9次缺页，其缺页率为 $9/12=75\%$ 。
- 过程中内存页面变化

|   | 1 | 2 | 3 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 4 | 4 | 4 | 5 | 5 | 5 | 5 | 5 | 5 |
|   |   | 2 | 2 | 2 | 1 | 1 | 1 | 1 | 1 | 3 | 3 | 3 |
|   |   |   | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 4 | 4 |



# 请求页式管理中的置换算法--FIFO

➤ 先进先出算法产生Belady现象的原因在于它根本没有考虑程序执行的动态特征。

# 请求页式管理中的置换算法—最久未使用

➤ 当需要淘汰某一页时，选择离当前时间最近的一段时间内最久没有使用过的页先淘汰。该算法的主要出发点是，如果某页被访问了，则它可能马上还要被访问。或者反过来说，如果某页很长时间未被访问，则它在最近一段时间也不会被访问。

# 请求页式管理中的置换算法—最久未使用

➤ 要完全实现LRU算法是一件十分困难的事情。因为要找出最近最久未被使用的页面的话，就必须对每一个页面都设置有关的访问记录项，而且每一次访问都必须更新这些记录。这显然要花费巨大的系统开销。因此，在实际系统中往往使用LRU的近似算法。

# 最久未使用页面置换算法

➤在需要淘汰某一页时，首先淘汰到当前时间为止，被访问次数最少的那一页。这只要在页表中给每一页增设一个访问计数器即可实现。每当该页被访问时，访问计数器加1，而发生一次缺页中断时，则淘汰计数值最小的那一页，并将所有的计数器清零。

最不经常使用  
页面淘汰算法

LFU

➤需要淘汰某一页时，从那些最近一个时期内未被访问的页中任选一页淘汰。只要在页表中增设一个访问位即可实现。当某页被访问时，访问位置1。否则，访问位置0。系统周期性地对所有引用位清零。当需淘汰一页时，从那些访问位为零的页中选一页进行淘汰。

最近没有使用页面淘汰  
算法

NUR

# 理想型淘汰算法

- 淘汰在访问串中将来再也不出现的或是在离当前页最远的位置上出现的页。
- 遗憾的是这种算法无法实现，因为它要求必须预先知道每一个进程的访问串。

# 存储保护

➤ 页式管理可以为内存提供两种方式的保护。  
一种是地址越界保护，另一种是通过页表控制  
对内存信息的存取操作方式以提供保护。

# 存储保护

- 地址越界保护可由地址变换机构中的控制寄存器的值——页表长度和所要访问的虚地址相比较来完成。
- 存取控制保护的实现则是在页表中增加相应的保护位即可。

# 页式管理的优缺点



- 由于它不要求作业或进程的程序段和数据在内存中连续存放，从而**有效地解决了碎片问题**。
- 动态页式管理提供了**内存和外存统一管理的虚存实现方式**，使用户可以利用的存储空间大大增加。这既**提高了主存的利用率**，又有利于组织多道程序执行。



# 页式管理的优缺点

- 要求有相应的**硬件支持**。例如地址变换机构，缺页中断的产生和选择淘汰页面等都要求有相应的硬件支持。这增加了机器成本。
- 增加了系统开销，例如缺页中断处理等。
- 请求调页的算法如选择不当，有可能产生**抖动现象**。
- 虽然消除了碎片，但**每个作业或进程的最后一页内总有一部分空间得不到利用**。如果页面较大，则这一部分的损失仍然较大。

# 练习题

➤某系统的空闲分区如表所示，采用可变式分区管理策略（动态分区管理策略），现有如下作业序列：96KB、20KB、200KB。若用最先适应算法和最佳适应算法来处理这些作业，则哪一种算法可满足该作业序列请求，为什么？

| 分区号 | 大小    | 起始地址  |
|-----|-------|-------|
| 1   | 32KB  | 100KB |
| 2   | 10KB  | 150KB |
| 3   | 5KB   | 200KB |
| 4   | 218KB | 220KB |
| 5   | 96KB  | 530KB |

# 练习题

采用首次适应算法时，96KB 大小的作业进入 4 号空闲分区，20KB 大小的作业进入 1 号空闲分区，这时空闲分区如下表所示。

| 分区号 | 大小    | 起始地址  |
|-----|-------|-------|
| 1   | 12KB  | 120KB |
| 2   | 10KB  | 150KB |
| 3   | 5KB   | 200KB |
| 4   | 122KB | 316KB |
| 5   | 96KB  | 530KB |

此时再无空闲分区可以满足 200KB 大小的作业，所以该作业序列请求无法满足。

# 练习题

采用最佳适应算法时，作业序列分别进入 5、1、4 号空闲分区，可以满足其请求。分配处理之后的空闲分区表见下表：

| 分区号 | 大小   | 起始地址  |
|-----|------|-------|
| 1   | 12KB | 120KB |
| 2   | 10KB | 150KB |
| 3   | 5KB  | 200KB |
| 4   | 18KB | 420KB |

# 练习题

➤ 页式存储管理，允许用户编程空间为32个页面（每个页面1KB），主存为16KB，如有一用户程序有10页长，且某一时刻该用户程序页表如右所示。如果分别遇到以下三个逻辑地址：0AC5H、1AC5H、3AC5H处的操作，试计算说明存储管理系统如何处理。

页面大小为1KB，所以低10位为页内偏移地址；用户编程空间为32个页面，即逻辑地址高5位为虚页号；主存为16个页面，即物理地址高4位为物理块号

| 逻辑页号 | 物理块号 |
|------|------|
| 0    | 8    |
| 1    | 7    |
| 2    | 4    |
| 3    | 10   |

# 练习题

➤答：逻辑地址0AC5H 转换为二进制为  
000 1010 1100 0101B，虚页号为  
2(00010B)，映射至物理块号4，故系统访  
问物理地址 12C5H(01 0010 1100 0101B)。

| 逻辑页号 | 物理块号 |
|------|------|
| 0    | 8    |
| 1    | 7    |
| 2    | 4    |
| 3    | 10   |

# 练习题

➤答：逻辑地址1AC5H 转换为二进制为 001 1010 1100 0101B ， 虚 页 号 为 6(00110B)，不在页面映射表中，会产生缺页中断，系统进行缺页中断处理。

➤逻辑地址3AC5H 转换为二进制为 011 1010 1100 011B，页号为14，而该用户程序只有10页，故系统产生越界中断。

| 逻辑页号 | 物理块号 |
|------|------|
| 0    | 8    |
| 1    | 7    |
| 2    | 4    |
| 3    | 10   |

O S

1

存储管理的功能

2

分区存储管理

3

页式管理

4

段式与段页式管理

5

覆盖与交换技术



# 分区式管理和页式管理

- 分区式管理和页式管理的进程地址空间都是**线性的**。
- 要求对源程序进行**编译链接**时，把源程序中的**主程序、子程序和数据区**等按**线性的一维地址空间**排列起来。
- 第一：这使得不同作业或进程之间**共享公用子程序和数据**变得非常困难（系统不能把用户和给定的**程序名和数据名**与这些被共享程序和数据在某个进程中的虚页对应起来）。

# 分区式管理和页式管理

- 第二：由于在页式管理时，一个页面中可能装有两个不同子程序段的指令代码，因此通过页面共享来达到共享一个逻辑上完整的子程序或数据块是不可能的。
- 第三：分区管理和页式管理只能采用静态链接。一个大的进程可能包含数百个甚至上千个程序模块，对它们链接要花费大量的CPU时间，而实际执行时可能只用到其中的一个子集

# 段式管理

➤基本思想——把程序按内容或过程（函数）关系分成段，每段有自己的名字。一个用户进程或作业所包含的段对应一个二维地址虚拟空间，也就是一个二维虚拟存储区。以段为单位分配内存，然后通过地址映射机构把段式虚拟地址转换成实际的内存物理地址。



# 段式虚存空间

- 段式管理把一个进程的**虚地址空间**设计成**二维结构**，即**段号 $s$** 与**段内相对地址 $w$** 。段号与段号之间**无顺序关系**，段的长度是**不固定**的。
- 每个段定义一组**逻辑上完整的程序或数据**。例如，一个进程中的程序和数据可被划分为主程序段、子程序段、数据段与工作区段。

# 段式虚存空间

➤每个段是一个首地址为零的、连续的一维线性空间。根据需要，段长可动态增长。

➤对段式虚地址空间的访问包括两个部分：段名和段内地址。

➤CALL [X] | <Y>                      LOAD 1, [A] | 6

STORE 1, [B] | <C>

# 段式管理的内存分配与释放

- 段式管理中以段为单位分配内存，**每段分配一个连续的内存区。**
- 由于各段长度不等，所以这些存储区的大小不一。
- 而且，同一进程所包含的各段之间不要求连续。

# 段式管理的内存分配与释放

➤ 段式管理的内存分配与释放在作业或进程的执行过程中动态进行。

- 首先，段式管理程序为一个进入内存准备执行的进程或作业分配部分内存，以作为该进程的工作区和放置即将执行的程序段。
- 随着进程的执行，进程根据需要随时申请调入新段和释放老段。进程对内存区的申请和释放可分为两种情况。

# 段式管理的内存分配与释放

1

➤ 当进程要求调入某一段时，内存中有足够的空闲区满足该段的内存要求。

2

➤ 当进程要求调入某一段时，内存中没有足够的空闲区满足该段的内存要求

# 内存中有足够的空闲区

➤可采用和动态分区式管理相同的空闲区管理方法。即把内存各空闲区按**物理地址**从低到高排列或按**空闲区大小**从小到大或从大到小**排列**。相对应的可采用**最先适应法**、**最佳适应法**和**最坏适应法**。

# 内存中没有足够的空闲区

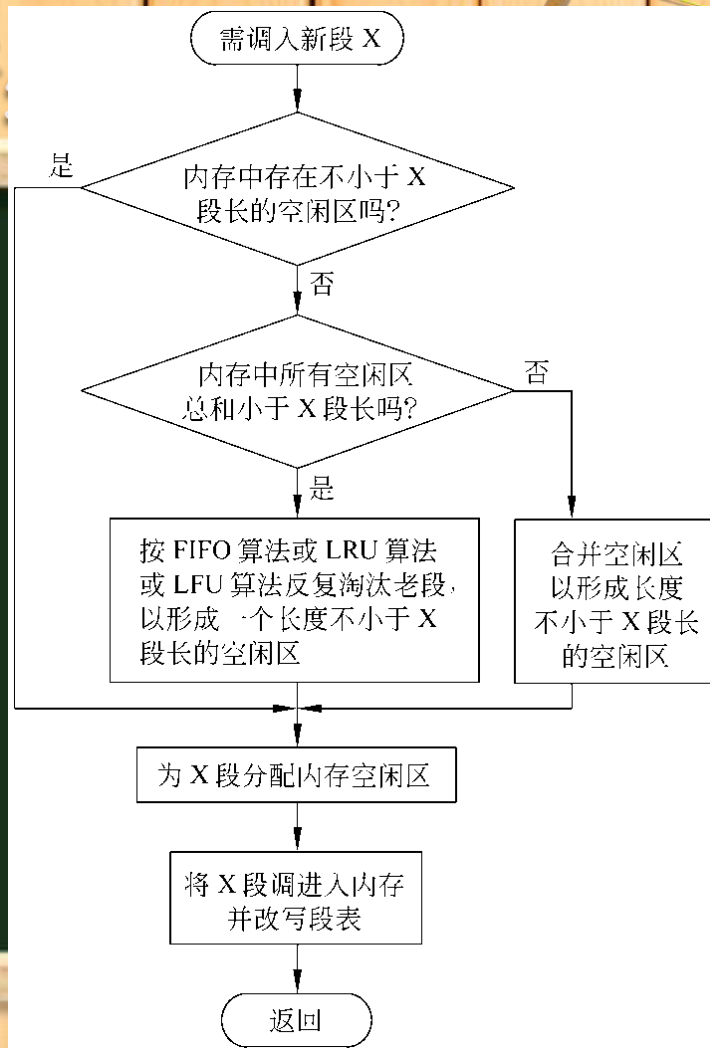
- 段式管理程序根据给定的**置换算法**淘汰内存中在今后的时间内不再被CPU访问的段，也就是**淘汰那些访问概率最低的段**。
- 动态页式管理中的几种常用的淘汰算法都可以用来作为段式管理时的淘汰算法。例如FIFO置换算法、LRU算法的近似算法等。

# 内存中没有足够的空闲区

➤但是，与页式管理时每页具有相同的长度时不一样，需要调入的某段长度可能大于被淘汰的一段程序或数据的长度。这样，仅仅淘汰一段可能仍然满足不了需要调入段的内存要求。此时，就应再淘汰另外的段直到满足需调入段的内存要求时为止。



# 缺段中断



# 段式管理的地址变换

➤ CPU如何感知到所要访问的段不在内存而启动中断处理程序呢？

➤ 段式地址变换机构解决虚拟空间。一个二维空间的虚拟地址怎样变换为一个一维的线性物理地址呢？



# 段式管理的地址变换

➤ **段表** (segment mapping table): 段式管理程

| 段号 | 始址 | 长度 | 存取方式 | 内外 | 访问位 |
|----|----|----|------|----|-----|
|    |    |    |      |    |     |
|    |    |    |      |    |     |
|    |    |    |      |    |     |
|    |    |    |      |    |     |
|    |    |    |      |    |     |

段表来进行内存管理的。

# 动态地址变换

段表地址寄存器

段表始址

段表

| 段号 | 始址   | 长度 | 存取方式 | 内外 | 访问位 |
|----|------|----|------|----|-----|
| 0  |      |    |      |    |     |
| 1  | 3400 |    | RW   | 内  |     |
|    |      |    |      |    |     |
|    |      |    |      |    |     |

3400



3520

虚拟地址

|   |     |
|---|-----|
| 1 | 120 |
|---|-----|

段号 段内地址

内存

# 段式管理的地址变换

- 段式管理时的地址变换过程也必须经过二次以上的内存访问。即**首先访问段表**以计算得到待访问指令或数据的物理地址，**然后才是对物理地址进行取数据或存数据操作**。
- 为了提高访问速度，页式地址变换时使用的高速**联想寄存器**的方法也可以用在段式地址变换中。

# 段式管理的地址变换

➤ 即高速联想寄存器中存放那些经常访问的段号所对应的段表项，且高速联想寄存器中的段表和内存的段表可同时查找。如果在联想寄存器中找到了所需要的段，则可以大大加快地址变换速度。

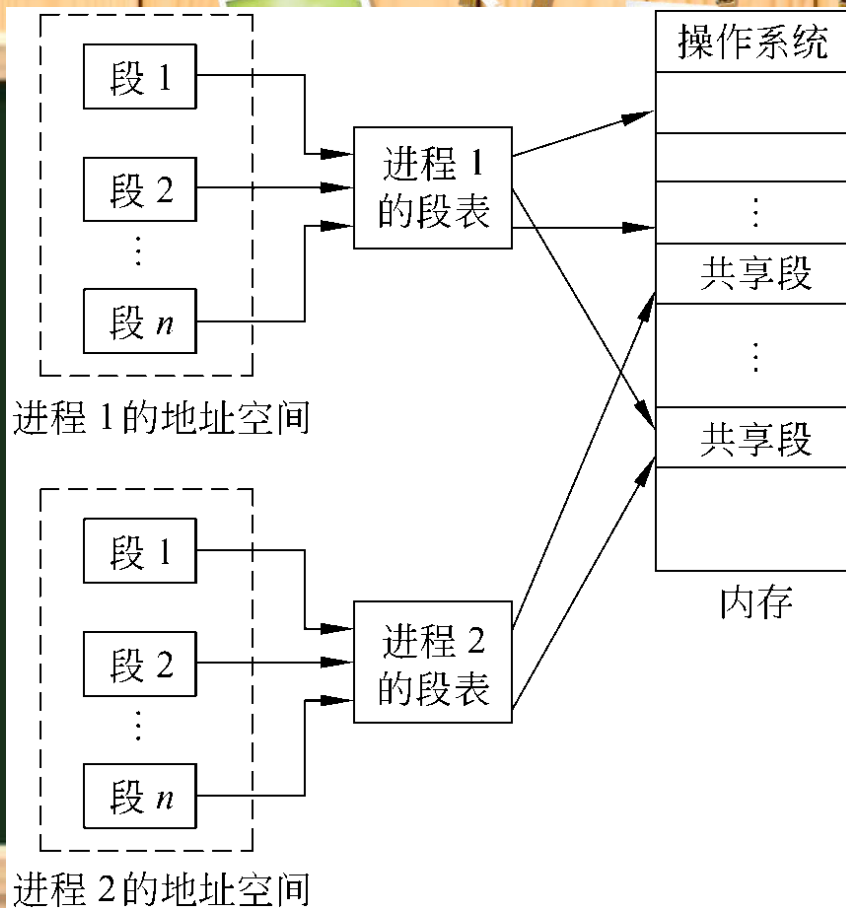


# 段的共享与保护

➤ 段式存储管理可以方便地实现内存信息共享和进行有效地内存保护。这是因为**段是按逻辑意义来划分的，可以按段名访问的缘故。**

# 段的共享

➤ 只要用户使用相同的段名，就可新的段表中填入已存在于内存之中的段的起始地址，并置以适当的读写控制权，就可做到共享一个逻辑上完整的内存段信息。



# 段的共享

➤另外，在多道环境下，由于进程的并发执行，一段程序为多个进程共享时，有可能出现**多次同时重复执行该段程序**的情况（即某个进程在未执行完该段程序之前，其他并发进程又已开始执行该段程序）。这就要求它在**执行过程中，该段程序的指令和数据不能被修改**。



# 段的共享

➤还有，与一个进程中的其他程序段一样，共享段有时也要被换出内存。这时，就要在段表中**设立相应的共享位来判别该段是否正被某个进程调用**。显然一个正在被某个进程使用或即将被某个进程使用的共享段是不应该调出内存的。

# 段的保护

➤段式管理的保护主要有两种。一种是地址越界保护法，另一种是存取方式控制保护法。关于存取方式控制保护已在前面介绍，这里不再重复。

# 段的保护

- 地址越界保护是利用段表中的**段长项**与**虚拟地址**中的**段内相对地址**比较进行的。若**段内相对地址**大于**段长**，系统就会产生**保护中断**。
- 不过，在允许段动态增长的系统中，段内相对地址大于段长是允许的。为此，段表中设置相应的**增补位**以指示该段**是否允许该段动态增长**。

# 段式管理的优缺点

- 段式虚存每次交换的是一段有意义的信息，而不是像页式虚存那样只交换固定大小的页从而需要多次缺页中断才能把所需信息完整地调入内存。
- 在段式管理中，段长可根据需要动态增长。这对那些需要不断增加或吸收新数据的段来说，将是非常有好处的。



# 段式管理的优缺点

- 便于对具有完整逻辑功能的信息段进行共享。
- 便于实现动态链接。由于段式管理是按信息的逻辑意义来划分段，每段对应一个相应的程序模块。因此，可用段名加上段入口地址等方法在执行过程中调入相应的段进行动态链接。



# 段式管理的优缺点

- 段式管理比其他几种方式要求有更多的**硬件支持**。这就**提高了机器成本**。
- 由于在内存空闲区管理方式上与分区式管理相同，在**碎片问题**以及为了消除碎片所进行的**合并等问题**上较分页式管理要**差**。

# 段式管理的优缺点

- 允许段的动态增长也会给系统管理带来一定的**难度和开销**。
- 段式管理的另一个缺点就是**每个段的长度受内存可用区大小的限制**。
- 和页式管理一样，段式管理系统在选择淘汰算法时也必须十分慎重，否则也有可能产生**抖动现象**。



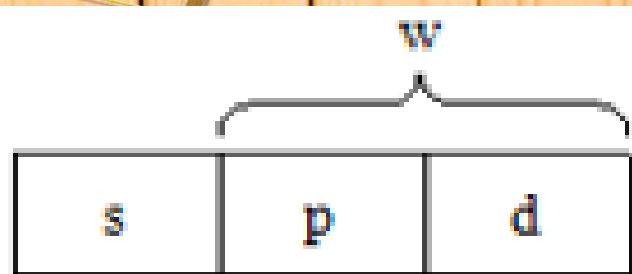
# 段页式管理的实现原理--虚地址的构成

- 段页式管理时，一个进程仍然拥有一个自己的二维地址空间。
- 首先，一个进程中所包含的具有独立逻辑功能的程序或数据仍被划分为段，并有各自的段号 $s$ 。这反映和继承了段式管理的特征。

# 段页式管理的实现原理--虚地址的构成

➤其次，对于段s中的程序或数据，则按照一定的大小将其划分为不同的页。和页式系统一样，最后不足一页的部分仍占一页。这反映了段页式管理中的页式特征。

# 段页式管理的实现原理--虚



- 段页式管理时的进程的虚拟地址空间中的虚拟地址，对于程序员可见的仍是段号s和段内相对地址w。
- 地址变换机构把w的高几位解释成页号p，以及把剩下的低位解释为页内地址d，因此整个虚拟地址由三部分组成：即段号s，页号p和页内相对地址d。

# 段页式管理的实现原理--虚地址的构成

➤ 由于虚拟空间的最小单位是页而不是段，从而内存可用区也就被划分成为若干个大小相等的页面，且每段所拥有的程序和数据在内存中可以分开存放。分段的大小也不再受内存可用区的限制。

# 段页式管理的实现原理—段页和段表

➤为了实现段页式管理，系统必须为每个作业或进程建立一张**段表**，管理内存分配与释放、缺段处理、存储保护和地址变换等。另外，由于一个段又被划分成了若干页，每个段又必须建立一张**页表**，把段中的虚页变换成内存中的实际页面。

## 段表地址寄存器

|      |      |
|------|------|
| 段表长度 | 起始地址 |
|------|------|

| 段号 | 其他 | 页表长度 | 起始地址 |
|----|----|------|------|
| 0  |    | 5    | 1024 |
| 1  |    | 7    | 1029 |
| 2  |    | 9    | 1036 |

段表

| 页号 | 其他 | 页面 |
|----|----|----|
| 1  |    | 12 |
| 2  |    | 19 |
| 3  |    | 21 |
| 4  |    | 8  |
| 5  |    | 10 |

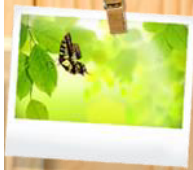
第 0 段页表

| 页号 | 其他 | 页面 |
|----|----|----|
| 1  |    | 29 |
| 3  |    | ⋮  |

第 2 段页表

|   |
|---|
|   |
| ⋮ |
|   |
|   |
| ⋮ |
|   |
|   |
| ⋮ |
|   |
|   |
| ⋮ |
|   |

内存



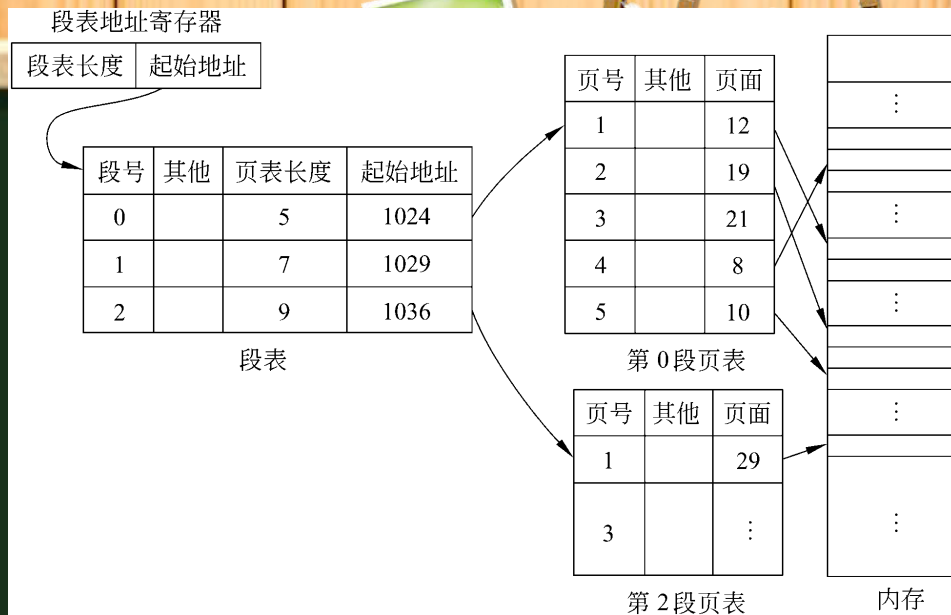
# 动态地址变换过程

- 在一般使用段页式存储管理的计算机系统中，都在内存中辟出一块**固定的区域**存放进程的**段表**和**页表**。
- 至少需要访问三次以上的内存。

# 动态地址变换过程

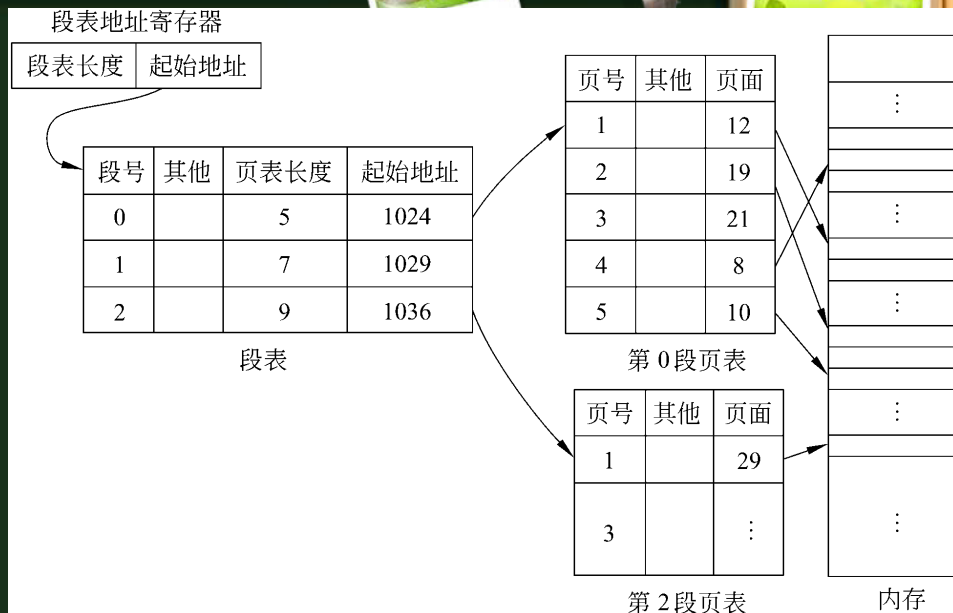
➤第一次是由段表地址寄存器得到段表始址去访问段表，由此取出对应段的页表地址。

➤第二次则是访问页表得到所要访问的物理地址。  
只有在访问了段表和页表之后



# 动态地址变换过程

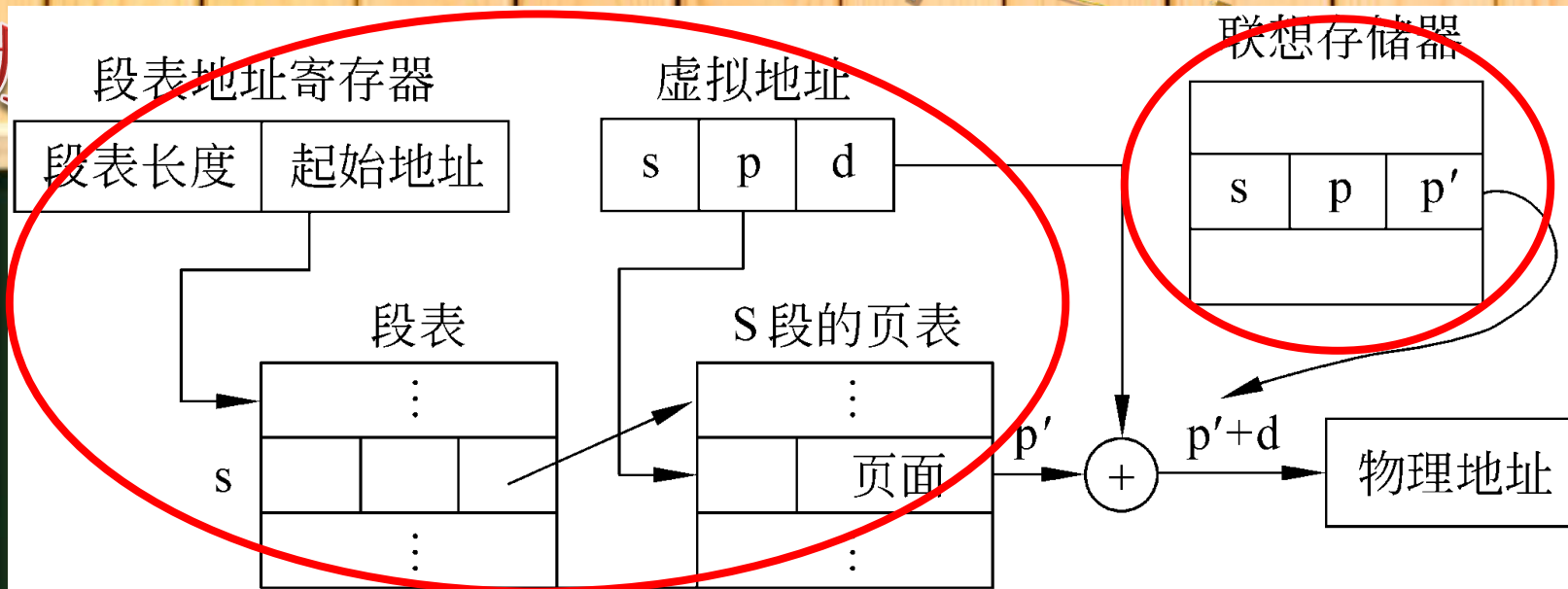
➤第三次才能访问真正需要访问的**物理单元**。  
显然，这将使CPU的执行指令速度大大降低。



# 动态地址变换过程

➤ 设置快速联想寄存器, 在其中存放当前最常用的段号 $s$ 、页号 $p$ 和对应的内存页面与其他控制用栏目。当要访问内存空间某一单元时, 可在通过段表、页表进行内存地址查找的同时, 根据快速联想寄存器查找其段号和页号。

动



➤经验表明，一个在快速联想寄存器中装有1/10左右的段号、页号及页面的段页式管理系统，可以通过快速联想寄存器找到90%以上的所要访问的内存地址。

# 动态地址变换过程

➤因为段页式管理是段式管理的页式管理方案结合而成的，所以具有它们二者的优点。但反过来说，由于管理软件的增加，复杂性和开销也就随之增加了。另外，需要的硬件以及占用的内存也有所增加。更重要的是，如果不采用联想寄存器的方式提高CPU的访内速度，将会使得执行速度大大下降。

# 实验六--- 作业调度模拟

## ➤实验目的:

■本实验要求学生模拟作业调度的实现，了解作业调度在操作系统中的作用。



# 实验六--- 作业调度模拟

## ➤实验要求:

- 1、预习作业调度的各种算法的思路;
- 2、完成实验报告, 交任课教师;
- 3、实验报告要求:
  - (1) 实验目的 (2) 程序所用的数据结构及其说明
  - (3) 打印一份源程序且附上注释。
  - (4) 打印程序运行前的作业表, 运行不同算法的作业调度程序, 打印每选择一个作业后的作业表变化情况以及作业被选中执行的次序。



# 实验六---作业调度模拟

## ➤实验内容:

- 采用先来先服务算法和运行时间最短者优先算法模拟设计作业调度程序。

# 实验七---请求页式存储管理中常用页面置换算法模拟

## ➤实验目的:

- 页式虚拟存储器实现的一个难点是设计页面调度（置换）算法，即将新页面调入内存时，如果内存中所有的物理页都已经分配出去，就要按某种策略来废弃某个页面，将其所占据的物理页释放出来，供新页面使用。
- 本实验的目的是通过编程实现几种常见的页面调度（置换）算法，加深读者对页面思想的理解

# 实验七---请求页式存储管理中常用页面置换算法模拟

## ➤实验要求:

- 1、熟练掌握页面请求算法的原理;
- 2、熟练掌握页面置换算法的思路;
- 3、上机前编写程序源码;
- 4、上机后, 写好实验报告和实验结果, 实验报告交任课教师。

# 实验七---请求页式存储管理中常用页面置换算法模拟

## ➤实验目的:

- 页式虚拟存储器实现的一个难点是设计页面调度（置换）算法，即将新页面调入内存时，如果内存中所有的物理页都已经分配出去，就要按某种策略来废弃某个页面，将其所占据的物理页释放出来，供新页面使用。
- 本实验的目的是通过编程实现几种常见的页面调度（置换）算法，加深读者对页面思想的理解

# 实验七---请求页式存储管理中常用页面置换算法模拟

## ➤实验内容:

- (1) 设计程序实现以上三种页面调度算法, 要求:
  - ①. 可以选择页面调度算法类型;
  - ②. 可以为进程设置分到物理页的数目, 设置进程的页面引用情况, 可以从键盘输入页面序列, 也可从文件中读取;
  - ③. 随时计算当前的页面调度次数的缺页中断率;
  - ④. 使用敲键盘或响应WM-TIMER的形式模仿时间的流逝;
  - ⑤. 以直观的的形式将程序的执行情况显示在计算机屏幕上;
  - ⑥. 存盘及读盘功能, 可以随时将数据存入磁盘文件, 供以后重复实验时使用。

# 实验七---请求页式存储管理中常用页面置换算法模拟

## ➤实验内容:

- (2) 假定进程分配到3个物理块, 对于下面的页面引用序列:
- 7-0-1-2-0-3-0-4-2-3-0-3-2-1-2-0-1-7-0-1
- 请分别用先进和先出调度算法, 最近最少用调度算法, 最近最不常用调度算法计算缺页中断次数, 缺页中断率和缺页调度次数、缺页置换率。
- 再假定进程分配到4、5个物理块, 重复本实验。

# 实验七---请求页式存储管理中常用页面置换算法模拟

## ➤实验内容:

- (3) 假定进程分配到3个物理块, 对于下面的页面引用序列:  
4-3-2-1-4-3-5-4-3-2-1-5-0-7-3-8-9-0-2-1-4-7-3-9
- 请分别用先进先出调度算法、最近最少用调度算法, 最近最不常用调度算法计算缺页中断次数, 缺页中断率和缺页调度次数、缺页置换率。
- 再假定进程分配到4、5个物理块, 重复本实验。

# 实验七---请求页式存储管理中常用页面置换算法模拟

## ➤ 实验内容:

■ (4) 假定进程分配到3个物理块，使用程序的动态页面序列生成算法，生成一个页面序列，将此序列存入磁盘文件。再从磁盘文件读入该序列，用程序分别计算三种算法下的缺页中断次数、缺页中断率和缺页调度次数、缺页置换率。

O S

1

存储管理的功能

2

分区存储管理

3

页式管理

4

段式与段页式管理

5

覆盖与交换技术

# 覆盖与交换技术

➤ 覆盖与交换技术是在多道环境下用来扩充内存的两种方法。

# 覆盖技术

➤覆盖技术的基本思想：一个程序并不需要一开始就把它的全部指令和数据都装入内存后再执行。单CPU系统中，每一时刻事实上只能执行一条指令。可以把程序划分为若干个功能上相对独立的程序段，按照程序的逻辑结构让那些不会同时执行的程序段共享同一块内存区。通常，这些程序段都被保存在外存中，当有关程序段的先头程序段已经执行结束后，再把后续程序段调入内存覆盖前面的程序段。这使得用户看来，好像内存扩大了，从而达到了内存扩充的目的。

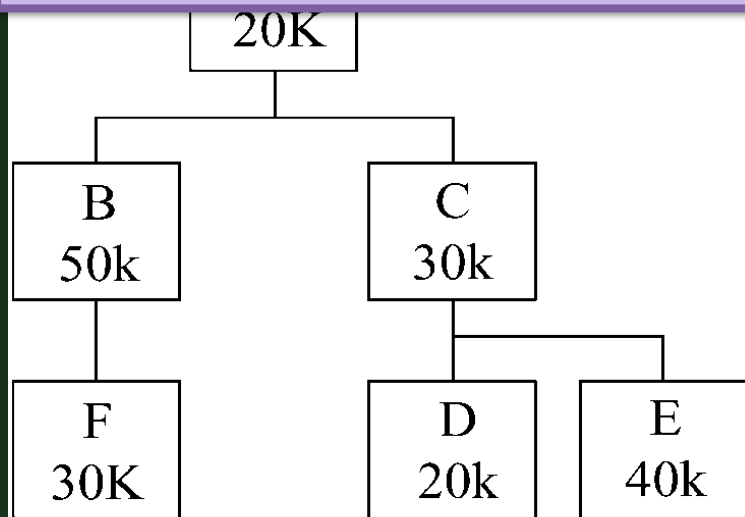


# 覆盖技术

- 覆盖技术要求程序员提供一个清楚的覆盖结构。即程序员必须完成把一个程序划分成不同的程序段，并规定好它们的执行和覆盖顺序的工作。
- 操作系统根据程序员提供的覆盖结构来完成程序段之间的覆盖。

# 覆盖技术

常驻内存部分与所有的被调用程序段有关，因而不能被覆盖。这一部分称为根程序。



(a)



(b)

# 交换技术

- 交换是指先将内存部分的程序或数据写入外存交换区，再从外存交换区中调入指定的程序或数据到内存中来，并让其执行的一种内存扩充技术。
- 交换不要求程序员给出程序段之间的覆盖结构
- 交换主要是在进程或作业之间进行。

# 交换技术

- 交换进程由**换出**和**换入**两个过程组成。
- 其中**换出** (swap out) 过程把**内存**中的数据和程序换到外存交换区。
- **换入** (swpin) 过程把**外存**交换区中的数据和程序换到内存分区中。

# 交换技术

SWAPOUT(i):

begin local m

m.base ← basei;

m.ceiling ← basei + sizei;

m.direction ← "out";

m.destination ← base of free area on swap area;

backupstorebasei ← m.destination;

send( (m, i), device queue);

end

分区的基址

分区的长度

方向

外存交换区中  
分区起始地址

用来记录被换出数  
据和程序的起始地  
址以便换入时使用

驱动设备做相应的数据读写操作，  
由交换进程发送给设备进程的消息  
m中应包含分区的分区号i

# 交换技术

SWAPIN(i):

begin local m

    m.base  $\leftarrow$  basei;

    m.ceiling  $\leftarrow$  basei + sizei;

    m.direction  $\leftarrow$  "in";

    m.source  $\leftarrow$  backupstorebasei;

    send( (m, i), device queue);

end

# 作业-1

➤ 在一个采用页式虚拟存储管理的系统中，有一用户作业，它依次要访问的字地址序列是：115，228，120，88，446，102，321，432，260，167，若该作业的第0页已经装入主存，现分配给该作业的主存共300字，页的大小为100字，请回答下列问题：

（1）按FIFO调度算法将产生 次缺页中断，依次淘汰的页号为 ，缺页中断率为 。

（2）按LRU调度算法将产生 次缺页中断，依次淘汰的页号为 ，缺页中断率为 。

## 作业-2

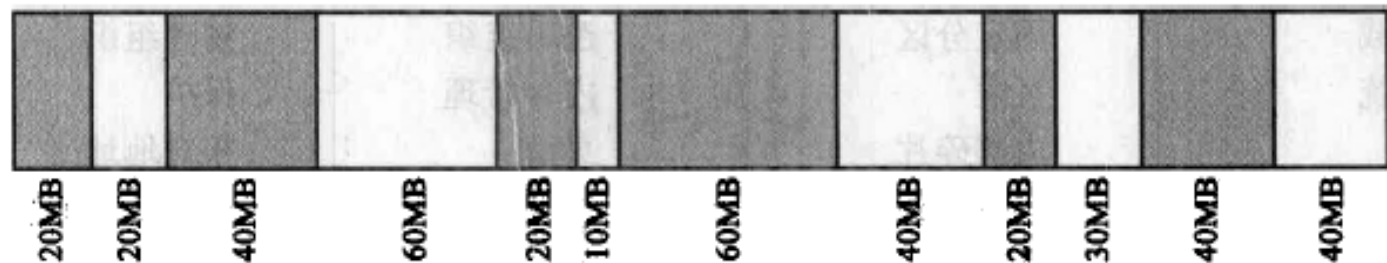
假设一个多道程序系统采用固定分区的内存管理策略，分区的大小分别为 Part0=20K，Part1=30K，Part2=30K，和 Part3=50K。同时假设下表中的进程需要调度进内存。假定这些进程都是（大致上）按照它们的进程号的顺序在同一时刻（时间 0）到达的。表格中的时间一栏表示这个进程需要分配一个分区来完成的时间，包含了完成该进程的全部时间（例如，交换时间，CPU 时间等等。）假设一个进程一旦被分配内存，该进程会一直驻留在内存中直到它完成为止（没有交换发生）。

a) 如果每一个进程都被分配最小的分区，且能够满足它的需求（在某些教材中被称为最佳适应分配策略），那么系统需要多长时间执行完表中所有进程。解释你的答案。

| 进 程 | 最大内存需求(K) | 时间(ms) |
|-----|-----------|--------|
| 1   | 18        | 10     |
| 2   | 29        | 5      |
| 3   | 10        | 20     |
| 4   | 33        | 15     |
| 5   | 14        | 15     |
| 6   | 49        | 10     |

## 作业-3

如果使用动态分区方案，下图所示为在某个给定的时间点的内存结构：



阴影部分为已经被分配的块；空白部分为空闲块。接下来的三个内存需求分别为 40MB、20MB 和 10MB。分别使用如下几种放置算法，指出给这三个需求分配的块的起始地址。

- a) 首次适配
- b) 最佳适配
- d) 最差适配

# 作业-4

在一个简单分段系统中，包含如下段表：

| 起始地址 | 长度（字节） |
|------|--------|
| 660  | 248    |
| 1752 | 442    |
| 222  | 198    |
| 996  | 604    |

对如下的每一个逻辑地址，确定其对应的物理地址或者说明段错误是否会发生：

- a) 0, 198
- b) 2, 156
- c) 1, 530
- d) 3, 444
- e) 0, 222

O S

谢谢大家！

THANK YOU!

